

Pedestrian detection with the A121 radar

How the feature extraction, the training and the embedded C pipeline fit together

SSAD lab project · FH Dortmund · 21 September 2026 · binary model: pedestrian / no pedestrian

Part	State	Where
Feature extraction (Python)	Done — 8 unit tests pin its behaviour	training/features.py
Binary model	Done — trained, evaluated as int8, exported	training/out/final/
Embedded C pipeline	Specified here, not written yet — ESP-IDF not installed; the radar-to-ESP32 link is still open	section 5

1 The idea in one paragraph

The radar produces 25 *frames* per second. Every frame is turned into one *column* of 64 numbers: how much energy is moving at each velocity, from 6.8 m/s towards the sensor to 6.8 m/s away from it. The last 64 columns — 2.56 seconds — form a 64×64 image, and a small CNN decides: pedestrian or not. On the ESP32-S3 this runs as a loop: new frame, new column, and every fourth frame one decision.

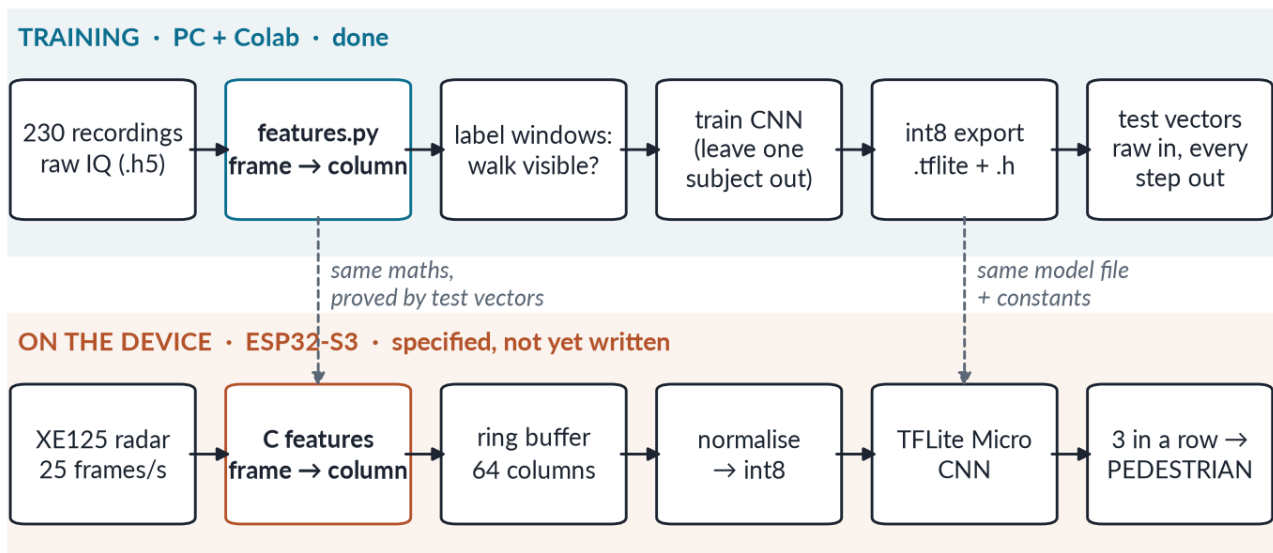


Figure 1 — The two halves of the project. The top row is done. The bottom row is what the ESP32-S3 has to do. The dashed links are the contract between them: the device must compute exactly what features.py computes, and it runs exactly the model file training exported.

The most important consequence of Figure 1: **the same maths runs in two places**. In training it runs in Python over the recordings; on the device it must run in C over live frames. If the two disagree, even slightly, the model sees inputs it was never trained on — and nothing crashes, the accuracy just quietly drops. Section 5 ends with the test vectors that prove the C code matches before anyone trusts an on-device number.

2 What the radar gives us

THE RADAR IN FIVE SENTENCES

The A121 sends very short 60 GHz pulses and measures each echo with its phase — a complex number, I and Q.

A **sweep** measures that echo at 22 **range points**: 1.00 m to 6.04 m in 24 cm steps.

A **frame** is 64 sweeps fired back to back in 11.6 ms (5,500 sweeps per second), and there are 25 frames per second.

Anything standing still returns the same echo in every sweep. Anything moving towards or away from the sensor shifts the echo's phase a little from sweep to sweep — the faster it moves, the faster the phase turns.

An FFT across the 64 sweeps turns *how fast the phase turns* into velocity: 64 velocity bins of 0.213 m/s, covering -6.81 to $+6.81$ m/s. In our recordings an approaching walker lands on the negative side (all 111 approach recordings).

Quantity	Value	Comes from
One frame	64 sweeps $\times$ 22 range points, complex int16 — 5,632 bytes	frozen config
Range	1.00 – 6.04 m, 24 cm per range point	start 400, step 96×2.5 mm
Frame rate	25 frames/s (one every 40 ms)	frozen config
Velocity bin	0.213 m/s; 64 bins span ± 6.81 m/s	60.5 GHz, 5,500 sweeps/s, 64 sweeps

3 Feature extraction: one frame becomes one column

`training/features.py` does these six steps for every frame. It is used for training, and it is also the specification for the C code, so every step is something a microcontroller can do in a loop: one frame at a time, no memory of earlier frames, no statistics computed on the fly, and nothing that depends on a label.

- 1 Remove what doesn't move.** For each range point, subtract the average of its 64 sweeps. Walls, floor and furniture give the same value in every sweep, so they vanish here. *Why*: static echoes can be far stronger than a person.
- 2 Window.** Multiply the 64 sweeps by a Hann window, $w[n] = 0.5 - 0.5 \cdot \cos(2\pi n/63)$. *Why*: without it, a strong return leaks into every velocity bin.
- 3 FFT.** A 64-point FFT across the sweeps, once per range point. Re-order the output so zero velocity sits at index 32 (an *fftshift*). Bin k is velocity $(k - 32) \times 0.213$ m/s.
- 4 Power.** Square the magnitude: $\text{re}^2 + \text{im}^2$.
- 5 Keep the two range points where things move most.** For each range point, add up the power in the *moving* bins — $|k - 32| \geq 3$, i.e. faster than 0.64 m/s. Keep the two range points with the most, and add their two spectra. *Why*: a person occupies one or two of the 22 range points. Summing all 22 adds the noise of twenty empty ones — in Figure 2(c) that costs the walker $12.6 - 4.8 \approx 8$ dB of contrast.
- 6 Logarithm.** $10 \cdot \log_{10}(\text{power} + 1\text{e-}6)$, giving 64 values in dB. That is one column.

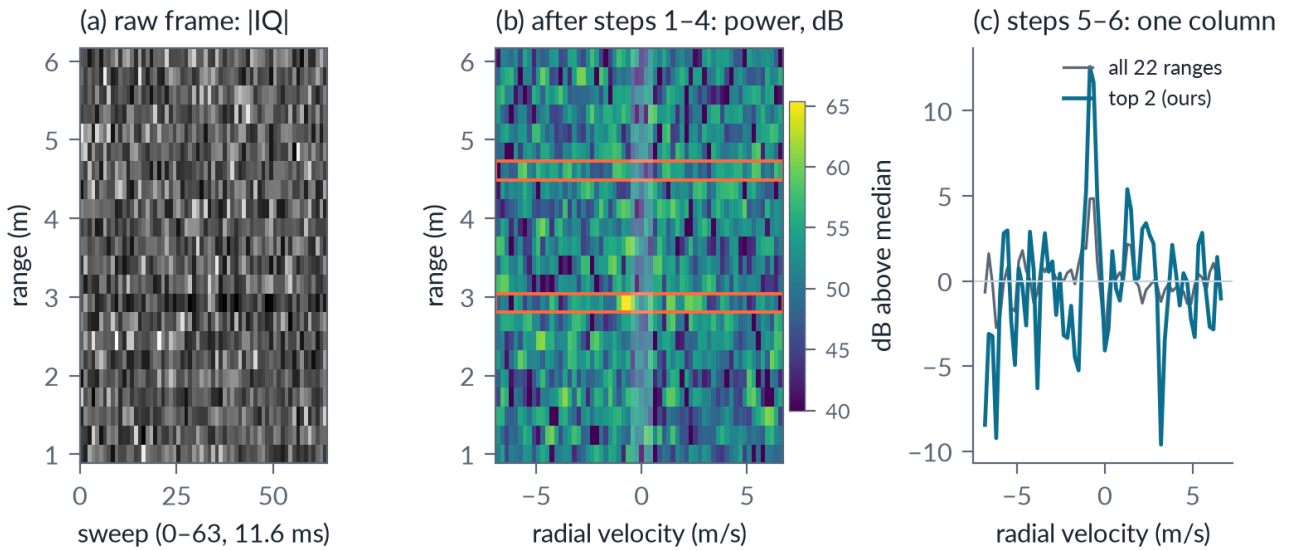


Figure 2 — One real frame, 10.3 s into `ped_approach_6m_normal_p1_take2.h5`. (a) The raw magnitudes: the walker is invisible. (b) After steps 1–4 the moving return appears near 3 m at -0.85 m/s; the pale band marks the slow bins step 5 ignores, and the orange rows are the two range points it keeps. (c) The finished column, against the naive alternative of summing every range point.

Stacking columns: the spectrogram

One column per frame, laid side by side, gives a *spectrogram*: time on one axis, velocity on the other. A walker is a band a little below zero — approaching at about 1 m/s. An empty room is texture only.

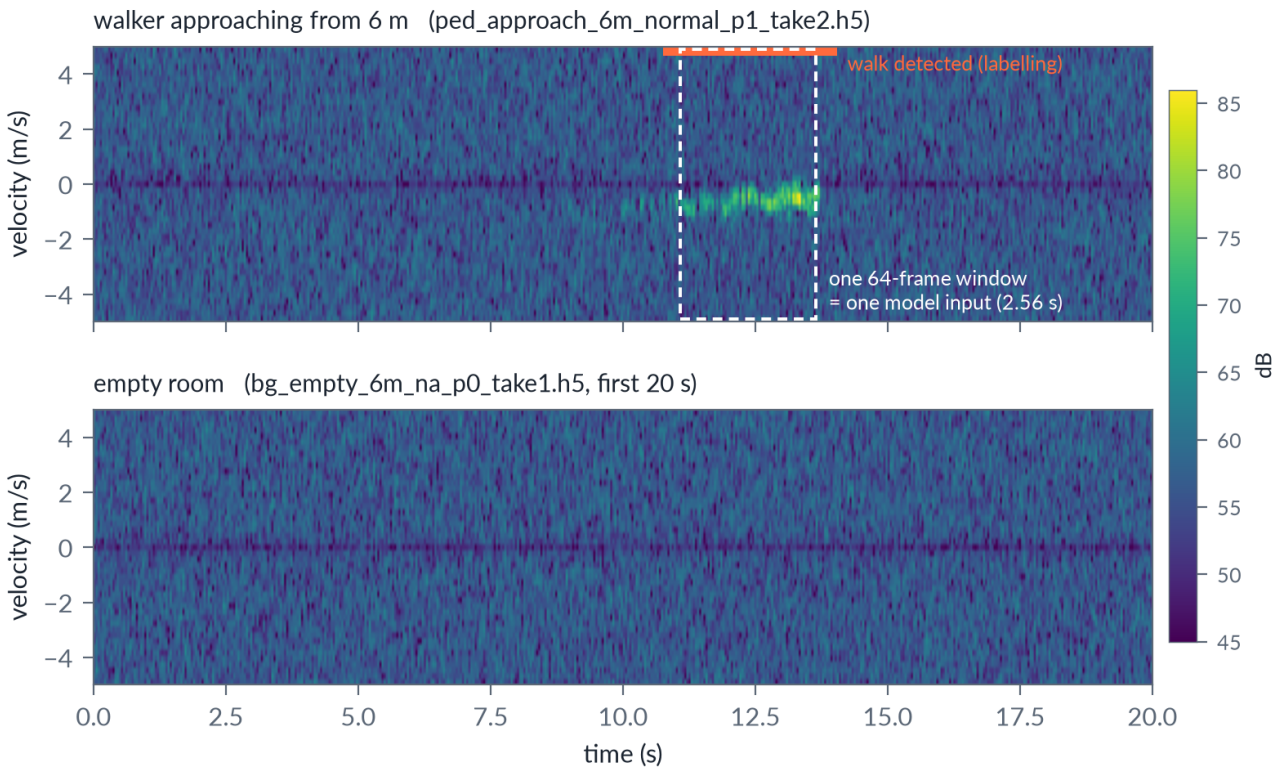


Figure 3 — Twenty seconds of two real recordings on the same colour scale. The orange bar marks where the labelling step (Section 4) found the walk; the dashed box is the size of one model input.

From columns to the model's input

The model sees 64 consecutive columns: a 64×64 image, oldest column first. Before that, every dB value is mapped to 0–1 with two fixed numbers chosen once during training:

$x = \text{clamp}((\text{dB} - 45.66) / (68.75 - 45.66), 0, 1)$. They are constants, not statistics of the current input, so the device needs no extra state — and, unlike the first notebook, nothing is scaled differently depending on the class.

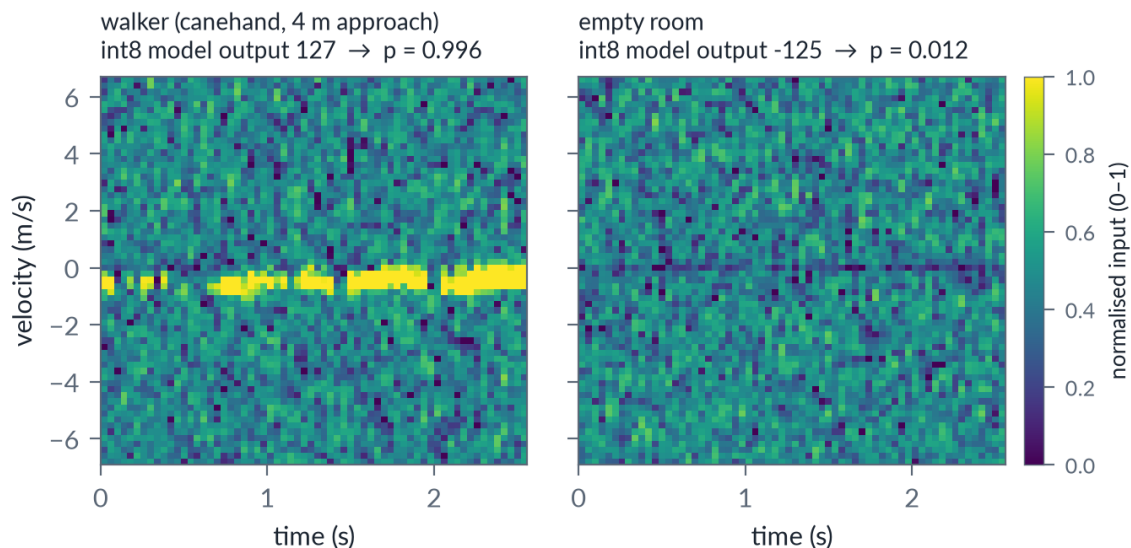


Figure 4 — The two test-vector windows exactly as the model receives them, with the int8 model's real outputs.

LOOK HOW FAINT THE WALKER IS

In its own range point a walker peaks only about 9–10 dB above the noise (5–7 dB in the _AP and _BW sessions), and the room's walls do not show up at all. The pilot recording from 15 August, with the same frozen settings, did show walls about 10 dB above the noise. Something about the setup changed before the campaign. Section 6 explains what that means for the model.

4 Training

Standard supervised training of a small CNN — the interesting decisions are in what counts as an example and how it is scored, so that is where this section spends its words.

Labels

Pedestrian = anyone walking, carrying something or not: the ped, bag, umbup, umbfwd, umbbck and canehand recordings (192). **No pedestrian** = every background recording (38), including a person standing still, which is background by the project's own class definition. Cane detection was dropped: a person with a cane is a pedestrian.

Which windows become training examples

A walk recording lasts about 20 s, but the walk itself takes a few seconds — the rest is an empty beam. Labelling every window of it *pedestrian* would teach the model that noise is a person. So a window only counts as pedestrian if at least one second of it shows visible walking.

'Visible' is decided once, by a simple detector: the moving power of the top two range points, smoothed over one second, compared with the recording's own median. The threshold is 0.549 dB — twice the loudest frame in all 38 background recordings, so no background frame ever crosses it. 179 of 192 walks pass; 13 are too faint to label. Windows from a walk recording that fail the test are *dropped*, never called background.

Windows overlap: a pedestrian window starts every 4 frames, a background window every 8. Each fold trains on 5,060–5,462 windows, 38–43 % pedestrian, with class weights to balance them.

The model

Layer	Output	Why
input	$64 \times 64 \times 1$	64 columns $\times$ 64 velocity bins, values 0–1
conv 3 $\times$ 3, stride 2, 16 ch + BatchNorm + ReLU	$32 \times 32 \times 16$	stride instead of pooling: fewer operations
conv 3 $\times$ 3, stride 2, 32 ch + BatchNorm + ReLU	$16 \times 16 \times 32$	
conv 3 $\times$ 3, stride 2, 48 ch + BatchNorm + ReLU	$8 \times 8 \times 48$	
global average pooling	48	fixed shapes; no reshape operators on the MCU
dropout 0.3 $\rightarrow$ dense, sigmoid	1	probability of a pedestrian

19,009 parameters, about 2.2 million multiply-adds per decision. BatchNorm folds into the convolutions at export, so the int8 file contains only four operator types: CONV_2D, MEAN, FULLY_CONNECTED, LOGISTIC. Adam at 1e-3, batch 64, early stopping on validation loss (patience 8), learning rate halved on plateaus. Augmentation: flip the velocity axis (approaching and departing are both pedestrians) and shift the level by up to ± 1 dB.

One trap worth knowing. With the default BatchNorm momentum of 0.99 the model looked perfect in training (98 %) while calling every validation window background: at about 80 steps per epoch the running statistics used at inference lag the batch statistics for about seven epochs, and early stopping locked in the broken weights. Momentum 0.9 fixes it from the first epoch. The code carries a comment so nobody reverts it.

How it was scored

Leave one subject out: train on two people, test on the third, three times. A detector that has met the person before is not the product. Background recordings are spread across the three folds by source.

Two scores per fold. The **window** score uses the clean windows only. The **recording** score runs the model over every window of every test recording — including the faint walks — exactly as the device would, and counts a detection when three windows in a row are positive. The recording score is the honest one: the window labels come from an energy threshold, so the window score mostly measures agreement with that threshold.

Test subject	Window acc.	Recall	False pos.	Walks found	Faint walks	False alarms
Bernard (p1)	99.1 %	98.3 %	0.32 %	58 / 58	5 / 6	2 in 10.5 min
Aish (p2)	99.6 %	98.9 %	0.00 %	60 / 61	2 / 3	0 in 10.5 min
Meghana (p3)	98.9 %	96.9 %	0.00 %	58 / 60	3 / 4	0 in 10.1 min
All three				176 / 179	10 / 13	2 in 31.1 min

All numbers are for the **int8** model that would ship; it agrees with the float model on at least 99.88 % of windows. The three missed walks are all session-one speed extremes (a fast 4 m approach, a slow 6 m approach, a fast diagonal). Both false alarms came from the recordings where a door opens and closes.

Export

The final model is trained on all three people (best validation epoch: 16) and converted to full-integer int8, calibrated on 400 windows drawn from *both* classes. The result is 24,976 bytes. Next to it: the same bytes as a C array, `radar_features_config.h` with every constant the device needs, and `test_vectors.npz`.

What changed from the first notebook

First notebook	Now
Background patches multiplied by 0.1 — the label leaked into the input, so background scored a trivial 100 %	No step depends on the label
Test set was the validation set, which had chosen the best epoch	Leave-one-subject-out; the test person is never seen
Windows cut from the middle of each recording; in 28 of 132 ped and cane-hand recordings the walk was elsewhere	Windows taken where the walk is actually visible
int8 calibrated on background only; quantised accuracy never measured	Calibrated on both classes; int8 scored in every fold
Normalisation constants not saved	Exported to radar_features_config.h
Model files that did not come from the notebook	Every file produced by one script, in one run

5 The embedded C pipeline — the specification

STATUS

Nothing in this section is written or tested on hardware yet. It is the specification the C code has to meet, built from features.py and the exported constants. ESP-IDF still has to be installed (TOOLCHAIN_SETUP.md), and how the XE125 hands frames to the ESP32-S3 is still open. The feature code is plain C; TensorFlow Lite Micro itself is C++.

The loop

Every 40 ms a frame arrives and becomes one column (Section 3). Columns go into a ring buffer holding the last 64. Every fourth frame — every 0.16 s — the 64 columns are normalised, quantised to int8 and handed to the model. One positive decision is not enough: a pedestrian is reported on the third positive in a row. That is the same rule the recording-level evaluation used, so its numbers carry over.

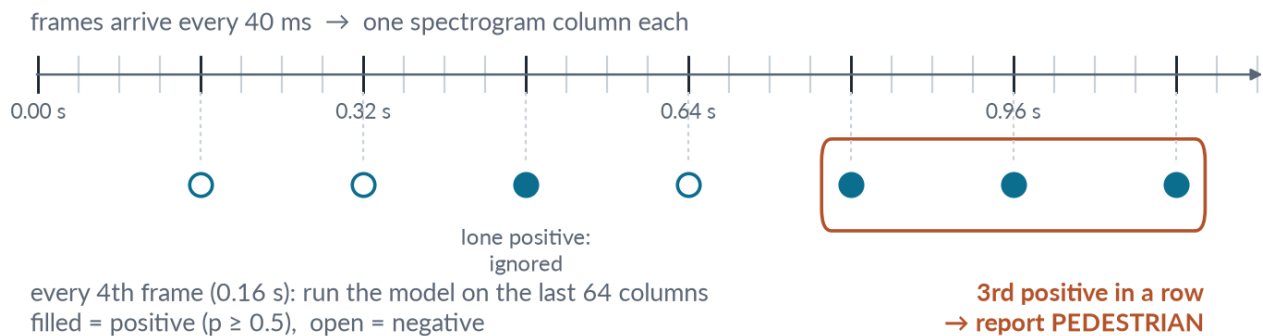


Figure 5 — Timing on the device. One column per frame, one decision per four frames, three positives in a row to report.

Per frame: steps 1–6 in C

```
/* Called once per frame (every 40 ms). iq = 64 sweeps x 22 range points, int16 I/Q.
 * Must reproduce features.py; check it against spec_db in test_vectors.npz. */
void frame_to_column(const int16_t iq[64][22][2], float col_db[64])
{
    static float power[22][64];          /* 5.6 KB */
    float moving[22];
    float buf[2 * 64];                   /* one complex FFT: re, im, re, im ... */

    for (int r = 0; r < 22; r++) {
        float mi = 0, mq = 0;            /* 1. remove what does not move */
        for (int s = 0; s < 64; s++) { mi += iq[s][r][0]; mq += iq[s][r][1]; }
        mi /= 64; mq /= 64;

        for (int s = 0; s < 64; s++) {    /* 2. Hann window */
            buf[2*s] = (iq[s][r][0] - mi) * HANN[s];
            buf[2*s + 1] = (iq[s][r][1] - mq) * HANN[s];
        }
        fft64(buf);                      /* 3. FFT, output in natural order */

        moving[r] = 0;
        for (int k = 0; k < 64; k++) {
            int j = (k + 32) & 63;        /* 3. fftshift: zero velocity -> index 32 */
            float p = buf[2*j]*buf[2*j] + buf[2*j+1]*buf[2*j+1]; /* 4. power */
            power[r][k] = p;
            if (abs(k - 32) >= RADAR_MOTION_MIN_BIN) moving[r] += p;
        }
    }
    int a = 0, b = 1;                    /* 5. two range points that move most */
    if (moving[1] > moving[0]) { a = 1; b = 0; }
    for (int r = 2; r < 22; r++) {
        if (moving[r] > moving[a]) { b = a; a = r; }
        else if (moving[r] > moving[b]) { b = r; }
    }
    for (int k = 0; k < 64; k++)          /* 6. dB */
        col_db[k] = 10.0f * log10f(power[a][k] + power[b][k] + RADAR_LOG_EPS);
}
```

Every fourth frame: normalise, quantise, decide

```
static int8_t ring[64][64];             /* last 64 columns, already normalised + quantised: 4 KB */
static int head = 0, count = 0, run = 0;

void on_frame(const int16_t iq[64][22][2])
{
    float col[64];
    frame_to_column(iq, col);
    for (int k = 0; k < 64; k++) {        /* normalise and quantise once, on arrival */
        float x = (col[k] - RADAR_DB_FLOOR) / (RADAR_DB_CEIL - RADAR_DB_FLOOR);
        x = fminf(fmaxf(x, 0.0f), 1.0f);
        int q = (int)lroundf(x / RADAR_IN_SCALE) + RADAR_IN_ZERO_POINT;
        ring[head][k] = (int8_t)(q < -128 ? -128 : q > 127 ? 127 : q);
    }
    head = (head + 1) & 63;
    if (++count < 64 || count % 4 != 0) return;

    int8_t *in = interpreter->input(0)->data.int8; /* row = time, oldest first */
    for (int t = 0; t < 64; t++) memcpy(in + 64 * t, ring[(head + t) & 63], 64);
    interpreter->Invoke();
    float p = (interpreter->output(0)->data.int8[0] - RADAR_OUT_ZERO_POINT) * RADAR_OUT_SCALE;

    run = (p >= RADAR_DECISION_THRESHOLD) ? run + 1 : 0;
    if (run == RADAR_EVENT_RUN) report_pedestrian(); /* once per run of positives */
}
```

Normalising and quantising each column *as it arrives* gives exactly the same model input as doing it per window — both steps work value by value with fixed constants — so the ring buffer can hold int8 (4 KB instead of 16 KB) and building the input becomes 64 small copies.

Memory and compute budget

Item	Size / cost	Note
One raw frame	5,632 B	64×22 complex int16
Power spectra, all range points	5,632 B	22×64 float, needed until step 5 picks two
FFT buffer	512 B	one 64-point complex FFT, reused per range point
Ring buffer	4,096 B	64×64 int8 (16 KB if kept as float)
Model, in flash	24,976 B	pedestrian_binary_int8.h
Tensor arena, in RAM	≈ 25 KB + overhead	widest step: 16 KB + 8 KB of activations — measure it
FFTs	550 per second	22 per frame $\times 25$ frames/s
Model	2.2 M MAC per decision	6.25 decisions/s ≈ 14 M MAC/s — time it on the S3

Things that break silently

- **FFT scaling.** NumPy's FFT does not divide by N . If the embedded FFT scales by $1/N$, every power value drops by 36.1 dB, everything normalises to zero, and the model says 'no pedestrian' forever. The test vectors catch it.
- **FFT sign.** The opposite sign convention flips the velocity axis. The model was trained with flips so it would mostly cope, but the test vectors will not match — fix it rather than rely on that.
- **Output order.** Radix-2 FFTs often return bit-reversed order; reorder before the fftshift ($\text{out}[k] = X[(k + 32) \bmod 64]$).
- **Hann endpoints.** Symmetric window, divide by 63 not 64; both ends are exactly zero.
- **Input layout.** Row = time with the oldest column first, column = velocity; element $t \cdot 64 + k$.
- **Rounding.** NumPy rounds halves to even, lroundf away from zero: an occasional ± 1 difference in the int8 input is expected and harmless.

Proving the C code matches

`test_vectors.npz` holds two real windows — a walker and an empty room — as raw sensor data, plus every intermediate the C code must reproduce. Check them in this order; each step only means something once the one before passes.

Array	Shape	The C code must...
<code>raw_real, raw_imag</code>	$2 \times 64 \times 64 \times 22$ int16	...read these as its input: 64 raw frames per window
<code>spec_db</code>	$2 \times 64 \times 64$	...produce these with <code>frame_to_column</code> , within 0.05 dB
<code>normalised</code>	$2 \times 64 \times 64$	...match these 0–1 values before quantising
<code>input_int8</code>	$2 \times 64 \times 64$	...match exactly (± 1 at rounding ties)
<code>output_int8</code>	2	...get 127 (walker) and –125 (empty room), or within a step or two
<code>probability</code>	2	0.996 and 0.012

Still open

- How the XE125 delivers frames to the ESP32-S3: which firmware on the module, which protocol, which wires. The 24 MHz logic analyzer in the lab kit is for exactly this.
- The real tensor-arena size and the real time per decision — both to be measured on the board.
- The display: class, confidence and ideally the scrolling spectrogram.

6 What this model can and cannot claim

- **The recordings are close to noise.** Raw values are flat across all 22 range points, the walls never appear, and a walker peaks about 9–10 dB above the noise in its own range point. The pilot recording showed walls with the same settings, so the setup changed before the campaign. Check the mounting before recording more: pointed at a wall about 2 m away, the sensor should show a strong, steady peak.
- **It is effectively a motion detector.** Every background recording — trolley, fan, standing person, umbrella — was indistinguishable from noise, so the model never learned to reject them. It fires on a door, and it has not been shown a fan or trolley it could actually see.
- **One room, three people.** There is no background recording from 16 September, the day of the _AP and _BW sessions, so false alarms against that day's conditions are untested.
- **If the signal gets stronger, retrain.** The two dB constants (45.66 and 68.75) describe today's signal levels. After fixing the setup, rebuild the dataset and retrain — about one minute locally plus 18 minutes on Colab — and re-export the header and test vectors.

7 Where everything is

File	What it is
training/features.py	The feature extraction — and the specification for the C code
training/build_dataset.py	Recordings → training/out/binary_dataset.npz (local, about 1 minute)
training/train_binary.py	Evaluation, final model, exports (Colab, about 18 minutes)
tests/test_features.py	8 tests: Doppler bins, clutter removal, range selection, frame-by-frame equals batch
training/out/report.json	Every evaluation number in Section 4
training/out/final/	Model (.keras, int8 .tflite, C array), radar_features_config.h, test_vectors.npz

Re-running it

```
.venv/bin/python training/build_dataset.py --root data/SSAD_DATA_RECORDINGS-main --out training/out

colab new -s ssad-binary
for f in training/out/binary_dataset.npz training/out/test_vectors_raw.npz \
    training/features.py training/train_binary.py; do
    colab upload -s ssad-binary "$f" "/content/${basename $f}"
done
printf 'import sys\nsys.argv = ["train_binary.py"]\nexec(open("/content/train_binary.py").read())\n' \
    | colab exec -s ssad-binary --timeout 5400
# pack /content/out, colab download it, then ALWAYS:
colab stop -s ssad-binary
```